

Lab03_Blumtritt

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE; Para o arquivo de vôos: Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; Leia apenas 1 milhão de linhas por vez; Remova vôos em que o aeroporto de destino comece com a letra 1; Remova registros em que existam pelo menos uma coluna faltante; Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(webshot2)

airports <- read_csv("airports.csv", col_select = c("IATA_CODE", "CITY", "STATE", "LATITUDE"
```

Rows: 322 Columns: 5

```
-- Column specification -----
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
f_filtro_voos <- function(chunk, pos) {
  chunk %>%
    select(DESTINATION_AIRPORT, ARRIVAL_DELAY) %>%
    filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY)) %>%
    filter(!grepl("^1", DESTINATION_AIRPORT))
}

flights_dados <- read_csv_chunked(
  "flights.csv.zip",
  DataFrameCallback$new(f_filtro_voos),
  chunk_size = 1000000
)
```

```
-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
```

```

WHEELS_OFF = col_character(),
WHEELS_ON = col_character(),
SCHEDULED_ARRIVAL = col_character(),
ARRIVAL_TIME = col_character(),
CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.

```

```

estats_suf <- flights_dados %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_atraso = sum(ARRIVAL_DELAY),
    total_voos = n(),
    .groups = "drop"
  )

flights <- stats_suf %>%
  mutate(mean_delay = soma_atraso/total_voos) %>%
  select(DESTINATION_AIRPORT, mean_delay)

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

Identifique a coluna que é a chave na tabela flights; Identifique a coluna que é a chave na tabela airports; Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports? Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; Armazene a tabela resultante no objeto flights.

A coluna chave na tabela flights é a DESTINATION_AIRPORT A coluna chave na tabela airports é a IATA_CODE

```

missing_airports <- flights %>%
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

flights <- flights %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```

airports_per_state <- airports %>%
  filter(!is.na(STATE)) %>%
  group_by(STATE) %>%

```

```
summarise(num_airports = n()) %>%
  arrange(desc(num_airports))
print(airports_per_state)
```

```
# A tibble: 54 x 2
  STATE num_airports
  <chr>      <int>
1 TX          24
2 CA          22
3 AK          19
4 FL          17
5 MI          15
6 NY          14
7 CO          10
8 MN           8
9 MT           8
10 NC           8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

Carregue o pacote leaflet; Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico; Armazene o grafico em b) numa variável chamada this; Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
library(leaflet)

this <- leaflet(flights %>% filter(!is.na(LATITUDE), !is.na(LONGITUDE))) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    radius = ~sqrt(max(0, mean_delay) + 1) * 2,
    color = "red",
    stroke = FALSE,
    fillOpacity = 0.6
  )
this
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 16:46:45 -03"
```